AuditLane

A reproducible pre-audit cleanroom for the Arbitrum Audit Program

A cleanroom that turns an Arbitrum repo into a normalised, evidence-carrying package a whitelisted audit firm can open without reconstructing the build.

Milestone 1 of the AuditLane application to the Arbitrum Audit Program. Arbitrum Sepolia testnet only.

doom2quake · Dipankar Sarkar

The gap that burns the subsidy

A project that lands audit funding still hands the auditor a **raw repository**.

- The build works on one machine and not another. The test suite needs undocumented setup. The deploy scripts point at a stale address.
- The whitelisted firm spends the first days of a fixed engagement **reconstructing the build** before a single line is reviewed.
- That reconstruction is paid for out of a subsidy meant to buy **review**, not plumbing.

With **$10M in ARB** flowing through a fixed whitelist of firms, every avoidable setup hour is subsidy that bought no security.

What is missing

There is no standard, reproducible, evidence-carrying package a project produces **before** the firm starts.

So the firm cannot:

- open a build that already works,
- read a test suite that already ran, with its counts captured,
- or check a verdict anyone can **recompute** rather than one it is asked to believe.

> "This build reproduced" has to be a checkable artifact, not a line in a PDF.

Why Arbitrum

The verdict is **checkable on-chain**.

- AuditLane's manifest is content-addressed; its reproducibility verdict is a pure function whose seven reason codes and fixed order are mirrored **byte-for-byte** in `src/BuildRegistry.sol`.
- A project **commits** a manifest id on Arbitrum Sepolia. Anyone recomputes the same id from the published package and checks the two agree. The commitment is **write-once**.
- The timing is the program: the Audit Program is live, funded for a fixed window, routing rolling engagements now.

The manifest: a pure fold

`auditlane build` reads a repo into a spec and folds it into one content-addressed manifest:

```
manifest_id = sha256( canonical_payload(
  pinned toolchain, source hashes, captured test counts,
  per-contract: source hash, address, deployed bytecode hash ))
```

- **No I/O, no clock, no network.** Everything that varies is passed in, so the id is a function of evidence and nothing else.
- Sources and contracts are sorted before hashing, so repo order cannot change the id.
- Two runs over the same repo state produce a **byte-identical** id.

The check: fixed order, fail closed

`check()` returns the reason codes that apply, in a **fixed order**. Empty list = reproducible.

```
1  NO_TOOLCHAIN_PIN      unpinned compiler range
2  BUILD_FAILED          no sources captured
3  TESTS_FAILED          declared suite did not pass
4  SOURCE_HASH_MISMATCH  a contract's source is missing
5  BYTECODE_MISMATCH     deployed bytecode != built artifact
6  MISSING_DEPLOYMENT    no resolved on-chain deployment
```

The scanner never invents a green suite: counts come from a captured `auditlane.tests.json`. The manifest reports the counts that **actually ran**.

The hero moment: a tampered deployment

```
3. A tampered deployment. AuditLane fails the package closed.
  NOT REPRODUCIBLE  sample-vault
    code 5  deployed bytecode does not match the built artifact
```

A deployed contract whose on-chain bytecode does not match the built artifact is not a warning to read past. It is **reason code 5** and a `NOT REPRODUCIBLE` verdict.

The agent never emits a green manifest a failing check produced. That is the trust primitive, and `forge test` proves the on-chain reference agrees.

One spec, two implementations

`auditlane/manifest.py` (off-chain): the pure fold and the fail-closed check.

`src/BuildRegistry.sol` (on-chain): the same seven reason codes in the same fixed order; `manifestDigest` recomputes the id; `commit` writes it **once** (re-commit reverts).

The Python `test_check_*` cases are the **exact analogues** of the Solidity `test*Fails` cases, so a green `pytest` proves the two agree on every code and its order.

The Arbitrum seam is keyless offline by default; the live `eth_getCode` path gates on `AUDITLANE_RPC_URL` and refuses any chain id but Arbitrum Sepolia (421614).

Verified, not asserted

- **13 Solidity tests** (`forge test` , solc 0.8.24): every reason code, the fixed order, the write-once commitment, the keccak digest.
- **24 Python tests** (`pytest`), keyless and offline: manifest determinism, content addressing, the scanner on a real fixture repo, the agent's fail-closed guardrails.

```
$ forge test  -> 13 passed
$ pytest -q   -> 24 passed
```

Every reason code has a test that fails without the check that raises it.

Honest limits (stated plainly)

- **No mainnet**, none planned under this grant. Arbitrum Sepolia testnet only.
- **No deployment yet.** `BuildRegistry.sol` is not deployed anywhere; that is milestone 2.
- **No live-chain test.** The `eth_getCode` path is wired and gated, but tests run the offline fixture.
- **No users. No revenue. No audit. No partnership with the Arbitrum Foundation.**

Our substitute for traction is tested code and a candid scope map. See `docs/LIMITATIONS.md`.

Roadmap

1. **M1 (this):** cleanroom core + `BuildRegistry.sol` reference, tested. **Built & green.**
2. **M2:** deploy to Arbitrum Sepolia + on-chain commit path + live `eth_getCode`.
3. **M3:** subsidised audit of AuditLane itself, by a whitelisted firm; each finding pinned by a test.
4. **M4:** documented auditor-facing package format + open handoff spec for any Arbitrum project.

The durable contribution: a standard, reproducible, evidence-carrying handoff format
that makes every subsidised audit in the program start from a better place.